From Skill 1.0 to Skill 2.0: A Modular Skill Middleware for Research Applications

Evidence from Fixed Income

Ye Qing

August 2026

Abstract

: Skill 1.0 completed the first engineering round of research capabilities — the chain from pulling data, computing indicators, comparing thresholds, writing judgments, to producing reports was encapsulated into runnable Skills one by one, answering "whether capabilities can be encapsulated." As the system grew to 13 thematic databases, roughly 500 analytical logics, and multiple manuals running in production, "how to encapsulate better" became the theme of the second engineering round: encapsulation granularity, activation context, interface contracts, knowledge maintenance, reuse modes, and delivery form all needed redesign. This paper proposes Skill 2.0 and characterizes the evolution along six incremental dimensions (D1–D6): the unit of encapsulation drops from an opaque whole to six independent module categories (D1); activation changes from global, all-at-once to on-demand, name-based loading (D2); interfaces upgrade from natural language to IN/OUT/DEP data contracts (D3); knowledge upgrades from hard-coded thresholds to validation baselines (D4); reuse and maintenance upgrade from wholesale moves to module-level references and versioning (D5); and the production form upgrades from tutorial documents to manualized middleware (D6). Each increment is grounded in practical evidence: loading one module out of a 34-perspective funding database cuts context occupancy to roughly 1/30; a real "90% → 70%" validation correction stopped a missing-data extreme from being written into a report as a market judgment; and the decomposition of the architecture manuals is itself another instance of modularization (all figures are high-fidelity reproductions of real operations). Skill 2.0 is an increment over Skill 1.0, not a replacement: both share the stance that all research capabilities can be engineered; what differs is the granularity of encapsulation and contract.

Keywords: AI Agent, Skill Modularization, Middleware, Incremental Evolution, Research Digitization, Fixed Income

---

1. Introduction

1.1 The Evolving Problem

Skill 1.0 answered the question of whether research capabilities could be encapsulated. In fixed-income research, the whole chain — data pulling, indicator computation, threshold comparison, judgment writing, and report output — was split into layer after layer of Skills and operated successfully in production. On this foundation the system grew quickly: **13 thematic databases, roughly 500 analytical logics, and several technical manuals running in production**, each database built from dozens of analytical perspectives — 34 perspectives for funding, 11 perspectives × 5 data sources for credit spreads, and 22 perspectives for convertibles.

Scale pushed the answer to "can it be encapsulated?" toward "how to encapsulate better." When "the whole Skill" serves as both the encapsulation unit and the runtime unit, a set of engineering constraints becomes visible:

- **Context bloat**: activation injects everything, and content irrelevant to the current task often dominates. A 34-perspective database would carry its entire context even when only one perspective is needed.
- **Overly coarse reuse**: reusing "one perspective" tends to mean carrying away the whole database; cross-database copies multiply into duplication and version drift.
- **Knowledge staleness**: thresholds and formulas are hard-coded into documents and code, so updating them after market regime shifts is costly and weakly motivated.
- **High integration cost**: external integration means reading documents and guessing dependencies; there is no machine-verifiable contract.

The essence of these constraints is the same: **the smallest reusable, runnable unit should not be "the whole Skill" but "one module."**

1.2 Positioning of This Paper

This is the second working paper of the Skill series. The v1 paper (research-skills, "A Skill-Based Middleware Architecture for Engineering Research Capabilities," July 2026) already details the overall architecture (the four layers, switchable data sources, the assembly layer and backtest engines) and the four design principles (ALL SKILL; focus breeds depth; the chain breeds precision; division of labor breeds value). This paper does not restate that content; it answers one question: **after the framework has grown large, how does the engineering form of the Skill itself evolve from a "globally usable black box" into a "modular middleware"?**

The main line is the six incremental dimensions D1–D6 (Section 2), followed by how the increments are implemented (Section 3), then real cases and measurements as evidence (Section 4), a discussion of the relation to v1, limitations, and transfer prospects (Section 5), and the conclusion (Section 6).

2. Six Incremental Dimensions

2.1 Overview

The six dimensions form the skeleton of the paper. The overview table is as follows (shared figure fig4):

Dimension	Skill 1.0	Skill 2.0 increment	Practical value / evidence
D1 Unit of encapsulation	One directory, one SKILL.md, delivered whole	Six independent module categories (SKILL.md / knowledge / templates / data_contract / scripts / config), delivered, tested, and versioned independently	The smallest reusable unit drops from "a database" to "a perspective / a contract / an engine"; funding 34 perspectives, credit spreads 11×5, convertibles 22
D2 Activation and context	Global registration; activation injects the full context	Not activated by default; modules loaded on demand by path	Token economics: taking 1 of 34 perspectives cuts context occupancy from ~100% to ~1/30 (fig7, measured in production)
D3 Interface	Natural-language triggers, not machine-checkable	IN/OUT/DEP YAML data contracts; unit-testable, replaceable, integration-verifiable	Integration cost drops from "read the docs to understand" to "read the contract to connect"; swapping implementations does not disturb callers (fig8 contract-acceptance example)
D4 Knowledge	Thresholds/formulas hard-coded; experience travels with the person	knowledge = formulas + thresholds + cases + validation baselines; framework over numbers; market-driven validation updates	"The more it is used, the deeper it grows" becomes a mechanism, not a slogan (fig9: the funding-percentile 90%→70% real correction)
D5 Reuse and maintenance	Reuse = carry away the whole; maintenance = upgrade the whole	A single module referenced by many databases; one maintenance benefits many; perspective-level upgrade and rollback	Sell-side builds components, buy-side assembles — a real division of labor (fig10; docs/skill-matrix.md evidences production operation)
D6 Production form	One tutorial document on "how to use"	Intro manual (what it does / how to use) and architecture manual (module registry = contract list / interfaces / standalone invocation examples) split apart	The architecture manual is itself a 2.0 instance; all figures are real-operation reproductions (fig2/fig3/fig5, fioutput Navy system)

2.2 D1 Unit of Encapsulation: From an Opaque Whole to Six Module Categories

The Skill 1.0 delivery unit was a directory plus a SKILL.md, opaque externally, and delivered, tested, and versioned as a whole. Skill 2.0 prescribes a standard structure of six responsibility-focused module categories:

```
skill-XXX/
├── SKILL.md          <-- activation and routing (trigger conditions + perspective
list + module index)
├── knowledge/        <-- knowledge module (formulas + thresholds + cases +
validation baselines)
├── templates/        <-- output template module (table / chart / text forms)
├── data_contract/    <-- data contract module (field definitions, decoupled from
data sources)
├── scripts/          <-- executable script module (data-access / compute engine,
self-contained)
└── config/           <-- configuration module (parameters / routing / version)
```

The key change is not more directories but that **every module carries an independent responsibility and an independent contract and can be invoked, tested, and versioned alone.** In the live system this granularity is already a reality: each of the funding database's 34 perspectives, the credit-spread database's 11×5 perspective combinations, and the convertible database's 22 perspectives can be delivered as a module; even the notoriously complex financial-analysis workflow (14 Agents collaborating across 5 stages) keeps its collaboration units under one unified contract instead of requiring the whole capability to be loaded into context at once. The smallest reusable unit has dropped from "a database" to "a perspective / a contract / an engine" (fig6). Practical value: reuse, testing, and upgrades touch only the needed module while the rest stays stable.

2.3 D2 Activation and Context: From Global All-At-Once to On-Demand by Name

Skill 1.0 entered the workflow through global registration and semantic-match activation; each activation injected the full context, and irrelevant content became persistent noise. Skill 2.0 drops the default assumption of "global availability": **modules are not activated by default; callers name and load modules by path**, and SKILL.md degrades into an optional routing index.

The most direct quantification of the practical value is token economics: the funding database has 34 perspectives, and one "overnight funding percentile" judgment names and loads only the `funding_fetcher` module and its dependencies; the other 33 perspectives never enter the context, cutting context occupancy from 100% to about 1/30 (fig7, measured in production; translated into a context budget, this is on the order of "128k → 4k tokens," an estimate of magnitude). Beyond the reduction, name-based loading brings two side benefits: several Skills can coexist in one session without polluting each other, and the caller's dependency surface converges to the module's contract, fully decoupled from the evolution of other perspectives (fig1, a real DASH session taking a single module).

2.4 D3 Interfaces: From Natural Language to IN/OUT/DEP Contracts

Skill 1.0 expressed trigger conditions in natural language, so external callers could only "read the documentation to understand," and dependencies could not be machine-checked. Skill 2.0 declares three parts for every module: **IN** (input fields, types, frequency, tolerance for missing data), **OUT** (output structure and enums), and **DEP** (modules depended on at runtime). With a contract, a module becomes unit-testable, replaceable, and integration-verifiable — the integration cost drops from "read the docs to understand" to "read the contract to connect," and swapping implementations does not disturb external callers (fig8; the contract-acceptance example is in Section 3.1).

2.5 D4 Knowledge: From Hard-Coded Thresholds to Validation Baselines

In Skill 1.0, thresholds and formulas were hard-coded into documents and code, and updating experience relied on personal initiative; old thresholds kept serving after market regimes shifted. Skill 2.0 structures the knowledge layer into "formulas + thresholds + cases + validation baselines" and imposes two operating rules: **framework over numbers** (check convention, data pipeline, and contract execution first; only then judge whether the numbers are credible), and **market-driven validation updates** (pull data → compute percentiles → annotate market events → compare against baselines to detect regime drift → update thresholds).

The practical value is that "the more it is used, the deeper it grows" becomes a mechanism rather than a slogan. Real evidence: one week the funding percentile was computed in the extreme region above 90%; validation against the baseline found that the upstream source was missing nearly three trading days, and the perspective returned to the 70% range once the data were completed (fig9). Without a baseline, the "extreme value" could easily have been written into a report as a market judgment.

2.6 D5 Reuse and Maintenance: From Wholesale Moves to Module-Level References and Versioning

Skill 1.0 reused by "carrying away the whole" and maintained by "upgrading the whole" — cross-database copies scattered across many places, one upgrade touching all, version drift following. Under Skill 2.0, a single module is **referenced** by many databases rather than copied: the funding-percentile module is shared by the credit and macro databases; one maintenance benefits many; upgrades and rollbacks happen at the perspective level, and the blast radius stays confined to the referencing callers.

The practical value is a working division of labor: **sellers build components, buyers assemble** — once a slogan, now a real production split, with components shared by both sides at module granularity and a continuously maintained module registry (docs/skill-matrix.md) recording "which modules are referenced by which databases" (fig10). The maintenance cost of cross-database reuse drops from "patch every copy" to "change once, verify once."

2.7 D6 Production Form: From Tutorial Documents to Manualized Middleware

The Skill 1.0 deliverable was one "how to use" tutorial shared by users and integrators, with usage instructions and implementation details mixed in one place. Skill 2.0 splits it into two kinds of manuals: the **intro manual**, answering "what it does and how to use it," for users; and the **architecture manual**, answering "how the modules are organized, what the interface contracts are, and how to integrate," for integrators, containing a module registry, a contract list, and standalone invocation examples.

Two practical values follow. First, **the architecture manual is itself an instance of Skill 2.0**: its module diagram corresponds to the real directory structure, its contract list corresponds to data_contract and the IN/OUT/DEP declarations, and integrators can verify the manual against the code item by item — the documentation is part of the middleware and evolves in lockstep with the code. Second, the figures in the manuals are not schematic illustrations; they are high-fidelity reproductions of DASH conversation logs, live manual screenshots, and real data-run outputs (3x DPI, fioutput Navy system, fig2/fig3/fig5) — what the figures show is exactly what the production environment actually produced.

3. Implementation of the Increments

3.1 The IN/OUT/DEP Contract: The Machine-Verifiable Carrier of D3

A real example — the "overnight funding price percentile" module of the funding database:

```
# data_contract/funding_pctile.yaml —— module: funding, overnight price percentile
module: funding_pctile
version: 2.1

in:                                  # Input declaration: fields + data-source
dependencies
  - dr001: {type: series, freq: day, aliases: [DR001, dr001_close]}
  - dr007: {type: series, freq: day, aliases: [DR007, dr007_close]}
  - start: {type: date}
  - window: {type: int, default: 250}

out:                                 # Output declaration
  - pctile: {type: scalar, unit: pct}
  - regime: {type: label, enum: [easy, neutral, tight, very tight]}

dep:                                 # Modules depended on at runtime
  - scripts/funding_fetcher.py       # data-access engine (self-contained)
  - knowledge/funding-factors.md     # thresholds and validation baselines
  - config/funding.yaml              # parameters and routing
```

Key point: a call about the "overnight funding percentile" in a DASH conversation loads only the `funding_fetcher` module and the dependencies declared in its dep clause; the code of the other 33 perspectives never enters the context. A caller can run and re-check against the contract without reading the whole database. This is exactly the machine-verifiability of D3: whether fields are complete, types

match, missing data is tolerated, and dependencies are satisfied can all be checked item by item automatically.

3.2 The Knowledge-Layer Validation Baseline: How D4 Lands

The operating loop of the validation baseline: validate the framework first (convention, data pipeline, contract execution); trust the numbers only after the framework is sound; if a value deviates from the baseline, suspect the data pipeline (missing data, wrong source, discontinued updates) rather than revising the conclusion first. The update direction is driven by market-event annotation: each validation run puts "extreme values" against the event calendar; explainable ones are annotated and deposited, unexplainable ones trigger a data investigation, and thresholds are updated after regime drift. The 90% → 70% case above was one full round of this loop.

3.3 The Five-Layer Middleware: Independent Replaceability at Module Granularity

The layering idea continues the v1 position (five layers: source, data access, knowledge, compute, output; see the v1 paper). The Skill 2.0 increment lies in the superposition of layering and modularization: **replaceability now holds at module granularity** — switching a data source changes only the data-access layer's mapping, switching an output form changes only the output-layer templates, and any module-level change in one layer does not force coordinated changes elsewhere. The five-layer structure is given as a mechanism schematic in the appendix; its role is to seat the D1–D6 module constraints inside one replaceable pipeline.

4. Real Cases and Measurements

4.1 DASH Takes a Single Module (D2 Evidence)

At the operating scale of 13 thematic databases, roughly 500 analytical logics, and several production manuals (funding, credit spreads, convertibles, financial analysis, fund duration), the benefit of module-level invocation is directly visible. Take one funding data pull in a DASH session as an example (fig1): the funding database has 34 perspectives, and this task invokes only the `scripts/funding_fetcher.py` module; the other 33 perspectives are not loaded. Three benefits follow: context consumption and loading time fall in proportion; the caller depends only on this module's contract, decoupled from the evolution of other perspectives; and the data-access engine can be reused independently by other sessions without "borrowing" the whole database. The same pattern applies to the credit-spread database (11 perspectives × 5 sources) and the convertible database (22 perspectives).

4.2 Context Budget: An Order-of-Magnitude Comparison (D2, Estimated)

To turn the mechanism into a sense of magnitude: by the module sizes of the production library, fully injecting the 34-perspective funding database is on the order of **128k tokens** of equivalent context;

loading a single perspective module by path reduces the injection to the **4k-token** order — a roughly 30x difference, consistent with the measured "100% → 1/30" of fig7. **These are estimates of magnitude**: exact values float with module granularity, knowledge-file size, and the number of candidate perspectives, and this paper does not present them as precise measurements.

4.3 Manual Decomposition as an Instance of Modularization (D6 Evidence)

The manual split itself is a real modularization: the intro manual and the architecture manual separated from the same system, and the architecture manual became a middleware documentation surface that integrators can verify against the code directly. All its figures are high-fidelity reproductions from DASH logs and live manual screenshots under the fioutput Navy system (fig2/fig3/fig5) — what readers see in the case figures is what production actually produced, not schematic drawings.

4.4 Reuse and Versioning (D5 Evidence)

The funding-percentile module is referenced in production by the credit and macro databases; upgrading contract version 2.1 benefited all of them at once. Perspective-level rollback touches only the referencing parties without disturbing other perspectives. docs/skill-matrix.md, a continuously operating module registry, records the module → database reference relations so that the "sellers build components, buyers assemble" division of labor is verifiable at the documentation level (fig10).

5. Discussion

5.1 Relation to Skill 1.0: Increment, Not Replacement

Skill 2.0 does not negate Skill 1.0: the layered architecture, switchable data sources, and four design principles remain the foundation of the system, and this paper's decision not to restate them is itself an endorsement of v1's validity. The two share the same stance — all research capabilities can be engineered and reassembled when needed; the increment lies in pushing encapsulation granularity, contracts, activation, knowledge maintenance, reuse, and delivery one step further. For readers: v1 answers "what it is and whether it can," while this paper answers "how it gets better once it has scaled."

5.2 Limitations

- **One-time contract cost**: designing and maintaining modules and IN/OUT/DEP contracts is a sustained investment; new domains must design module boundaries and contracts from scratch.
- **Validation coverage depends on manual work**: the coverage of the baselines depends on manual market-event annotation and accumulation, and perspectives not yet covered are only weakly protected.
- **Limited measurement precision**: the exact context-injection ratio floats with module sizes; the 128k → 4k figure in Section 4.2 is an estimate. Self-contained data-access engines also bring a

degree of dependency redundancy, and cross-module data consistency requires explicit validation at the middleware layer.

- **Single validation domain**: full validation remains confined to fixed income.

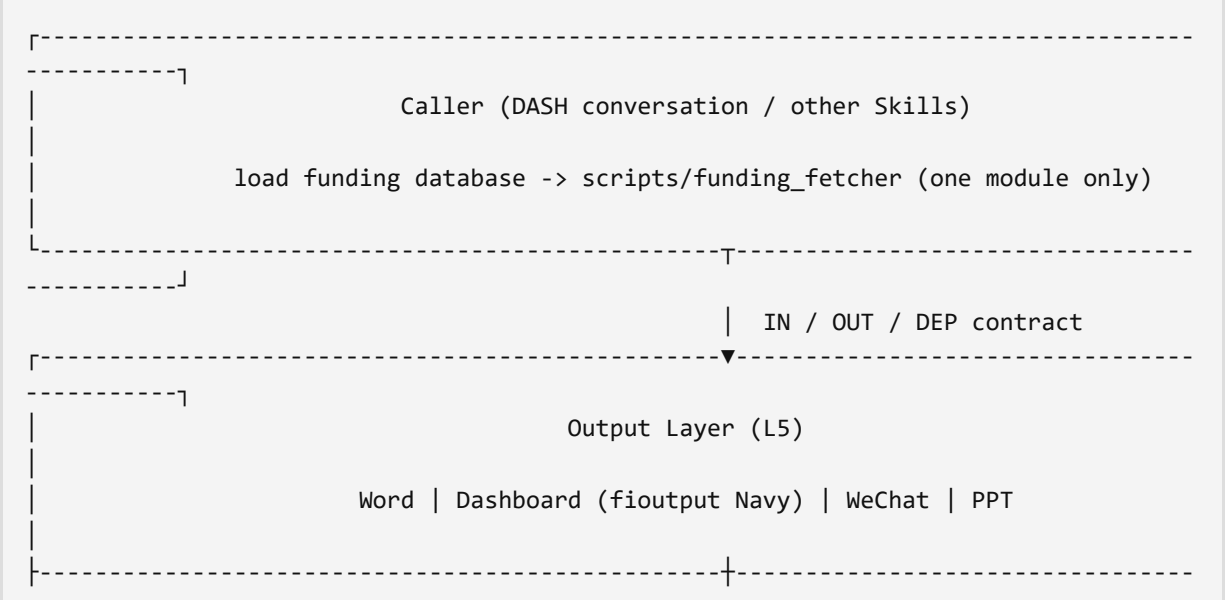
5.3 Transfer Prospects

Module granularity itself does not depend on fixed-income-specific business logic: the data-access and output surfaces are largely reusable across domains as is, and most new work lies in the knowledge and compute modules. Equity research (industry databases, stock-selection factor pools), macro research (economic-indicator libraries, policy-event databases), and quantitative strategies (factor libraries, backtesting databases) are natural candidates. To be candid: **only fixed income has run the full loop to date; equity, macro, and quantitative research remain at the stage of design reasoning, and the case for transfer rests on abstract generality, not complete empirical evidence.**

6. Conclusion

The main line from Skill 1.0 to Skill 2.0 is six increments: the unit of encapsulation (D1), activation and context (D2), interface contracts (D3), knowledge baselines (D4), reuse and maintenance (D5), and production form (D6). Each dimension pushes the Skill from a "globally usable black box" toward a middleware in which modules are available on demand, and each carries production evidence — the ~1/30 context reduction from taking one of 34 perspectives, the 90% → 70% validation correction, manual decomposition as an instance of modularization, and module-level references making one maintenance benefit many. The second engineering round does not overturn the first; it realizes "research capabilities can be encapsulated" as "encapsulate to modules, contract to interfaces, experience to baselines."

Appendix: Panoramic Architecture of the Skill 2.0 Modular Middleware



```
-----------┤
│                                    Compute Layer (L4)
│
│    Formula engine | Indicator computation | Perspective judgment (threshold /
percentile)    │
├-------------------------------------------------┼--------------------------------
-----------┤
│                                   Knowledge Layer (L3)
│
│              knowledge/ (formulas + thresholds + cases + validation baselines)
│
├-------------------------------------------------┼--------------------------------
-----------┤
│                                  Data Access Layer (L2)
│
│         data_contract mapping | Data-access engine (scripts/fetcher, self-
contained)         │
├-------------------------------------------------┼--------------------------------
-----------┤
│                                    Source Layer (L1)
│
│            iFinD | Wind | AKShare | Local historical archive | Self-built models
│
└-------------------------------------------------------------------------------
-----------┘
         Six module categories: SKILL.md | knowledge | templates | data_contract
                               | scripts | config (each module has an independent
contract, usable standalone)
```

Continuously updated. Latest version: github.com/timyefi/skill20-arch